Clasificación de identidad y emoción en rostros

Trabajo Práctico 1 — Grupo 8

IA y Aprendizaje Automático II · UCA Rosario · Noviembre 2026

Resumen

Se entrenaron clasificadores de imágenes para dos tareas sobre rostros de figuras públicas argentinas: *identidad* (20 personas) y *emoción* (alegría, enojo, neutral, sorpresa y tristeza). Se compararon cuatro CNNs diseñadas y entrenadas desde cero, tres modelos con transfer learning y tres baselines, en TensorFlow/Keras sobre una GPU T4 de Colab. En el test del dataset final (411 imágenes), ResNet50V2 —elegida por validación en las dos tareas— alcanza un F1 macro de 0,974 en identidad y 0,482 en emoción. La identidad no muestra sobreajuste; en emoción las CNNs desde cero sobreajustan y los modelos de transferencia quedan cortos aun en train, lo que apunta a un límite de los datos más que de las redes. Se analizó qué mira cada modelo (Grad-CAM), si la emoción se predice a través de la identidad y la calidad del dataset con un autoencoder. Los análisis muestran que la identidad se apoya en parte en el fondo del video, por lo que esas cifras están algo infladas. El modelo final se publicó en una aplicación web.

Índice

1. Objetivo **1**

2. Datos **1**

2.1. Descripción . . . 1

2.2. Partición sin copias entre train y test . . . 1

2.3. Relación persona–emoción . . . 2

3. Pipeline y entrenamiento **2**

3.1. Pipeline de datos . . . 2

3.2. Entrenamiento . . . 3

4. Arquitecturas **3**

4.1. CNNs desde cero . . . 3

4.2. Transfer learning . . . 5

4.3. Baselines . . . 5

5. Métricas **5**

6. Resultados **6**

6.1. Comparación de modelos . . . 6

6.2. Errores en emoción . . . 7

6.3. G8 dentro del test final . . . 7

7. ¿Sobreajuste o subajuste? **7**

8. ¿Qué aprende el modelo? **8**

8.1. Qué ve la CNN por dentro . . . 8

8.2. Atajo persona → emoción . . . 8

8.3. Grad-CAM: dónde mira la red . . . 9

8.4. Fuga de escena . . . 10

8.5. Calidad del dataset con un autoencoder . . . 10

9. Aplicación web **11**

10. Conclusiones **11**

1. Objetivo

El trabajo pide entrenar redes neuronales que, a partir de una foto de un rostro, respondan dos preguntas independientes: *¿quién es?* (20 clases) y *¿qué emoción muestra?* (5 clases). Cada grupo armó un conjunto de imágenes propio (G8: Dolores Fonzi y Pablo Echarri) y la cátedra las unió en un dataset final. Se buscó, además de un buen número, entender *por qué* funciona cada modelo, *dónde* se equivoca y *qué* está mirando, y detectar atajos que inflen los resultados.

La entrega es un notebook de TensorFlow/Keras que se corre de punta a punta en Colab con GPU T4: lee los datos, arma la partición, entrena todos los modelos (unos 35 minutos de entrenamiento en total), los evalúa y genera todas las figuras de este informe.

2. Datos

2.1 Descripción

- **Dataset final:** 2723 imágenes de 224×224 de 20 personas, extraídas de fotogramas de videos (entrevistas, programas, redes). **G8:** 221 imágenes de dos personas, incluidas también en el final.
- **Limpieza:** se buscaron duplicados exactos con md5. Hubo 10 imágenes repetidas con etiquetas en conflicto y se eliminaron; quedan 2713.
- **Desbalance:** por persona, de 63 (Julio Chávez) a 419 imágenes (Guillermo Francella); por emoción, de 340 (tristeza) a 765 (neutral). Un modelo que respondiera siempre "Francella" acertaría el 15 % sin aprender nada, por eso la métrica principal es el F1 macro (sección 5).

Emoción	Train	Val	Test	Total
neutral	538	114	113	765
alegría	436	99	99	634
enojo	352	76	76	504
sorpresa	327	72	71	470
tristeza	236	52	52	340
Total	1889	413	411	2713

Cuadro 1. Imágenes por emoción y partición (70/15/15).

2.2 Partición sin copias entre train y test

Las imágenes salen de videos, así que hay muchos fotogramas casi idénticos. Si uno queda en train y su "gemelo" en test, el modelo acierta por memoria y el test deja de medir generalización. Para evitarlo:

1. Se calcula un *hash perceptual* (phash) de cada imagen: una huella de 64 bits que cambia poco si la imagen cambia poco.
2. Dos imágenes de la misma persona con distancia de Hamming ≤ 8 entre sus huellas se consideran casi duplicadas y se unen en un grupo (el más grande tiene 69 fotogramas).
3. Los grupos se reparten enteros en train, validación y test (70/15/15, semilla 42), persona por persona, de modo que cada persona y cada emoción queden cerca del 15 % en test. El notebook verifica que ningún grupo quede partido.

La partición se guarda en un CSV con un hash de control, así cualquier corrida usa exactamente la misma.

2.3 Relación persona–emoción

Como las fotos de cada persona vienen de pocos videos, cada persona tiene su propia mezcla de emociones: Colapinto aparece sorprendido el 41 % de las veces, Azzaro triste el 40 %, Georgina Barbarossa neutral el 50 %. Para medir cuánto se relacionan las dos variables se usó la **V de Cramér**, que lleva el estadístico χ^2 de la tabla de contingencia persona $\times$ emoción a una escala de 0 (independientes) a 1 (la persona determina la emoción):

$$V = \sqrt{\frac{\chi^2}{n\,(\min(r, c) - 1)}} = \sqrt{\frac{667}{2713 \cdot 4}} = 0{,}248.$$

Es una relación moderada: saber quién es da pistas sobre la emoción. Esto abre un posible atajo (reconocer a la persona y responder su emoción habitual) que se controla en la sección 8.2.

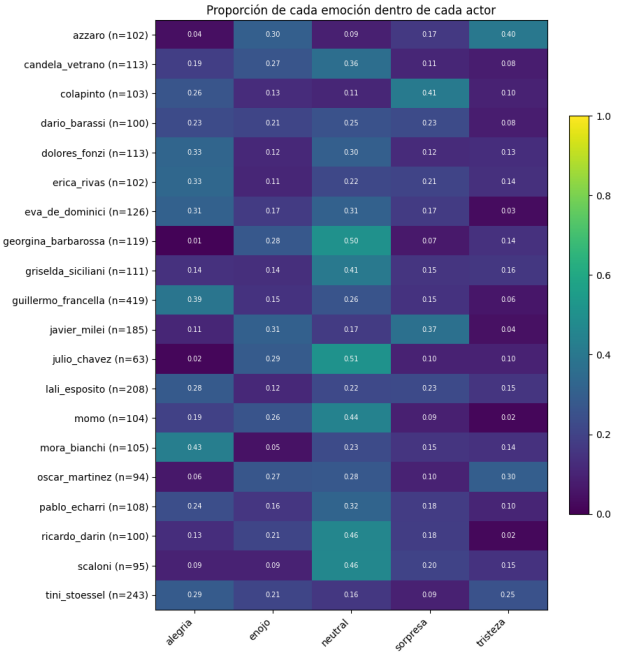


Figura 1. Proporción de cada emoción dentro de cada persona. Si no hubiera relación, todas las filas serían parecidas.

3. Pipeline y entrenamiento

3.1 Pipeline de datos

Se usa `tf.data`: las imágenes se decodifican una vez, se guardan en caché y se cargan por lotes de 32 con *prefetch*, para que la GPU no espere al disco. La aumentación se aplica solo en train y en cada época de forma distinta: espejo horizontal, rotación de $\pm 11°$, y brillo y contraste de ± 10 %. No se usan espejo vertical ni recortes o rotaciones fuertes: generarían caras invertidas o cortarían la boca y los ojos, que es justo donde está la emoción.



Figura 2. Cuatro imágenes de train (primera columna) y tres versiones aumentadas de cada una.

3.2 Entrenamiento

- **Pérdida:** entropía cruzada con **pesos por clase** $w_c = n/(K \cdot n_c)$: un error en tristeza (clase chica) pesa más que uno en neutral, para que el modelo no ignore las clases con pocas imágenes.
- **Early stopping** sobre la pérdida de validación, restaurando los pesos de la mejor época. Así el modelo que se guarda es el de mejor validación, aunque el entrenamiento haya seguido y empezado a sobreajustar.
- **ReduceLROnPlateau:** si la pérdida de validación se estanca, la tasa de aprendizaje se divide por 2.
- **Precisión mixta** (*float16* en la GPU, salida en *float32*): casi el doble de velocidad en la T4.
- **CNNs desde cero:** entrada de 128 px (suficiente para una cara y 3 veces menos cómputo que 224), Adam con tasa 10^{-3}, hasta 60 épocas, paciencia 15.
- **Transfer learning** (224 px, pesos de ImageNet) en dos fases. Fase 1: la base congelada y solo se entrena la cabeza nueva (GAP + dropout 0,3 + densa), hasta 15 épocas. Fase 2 (*fine-tuning*): se descongelan las últimas capas (30 en MobileNetV2, 20 en EfficientNetB0, 40 en ResNet50V2) con tasa 10^{-5}, hasta 25 épocas. Las capas de BatchNorm quedan congeladas: con lotes chicos, actualizar sus estadísticas destruye lo aprendido en ImageNet.
- Toda decisión (modelo, época, hiperparámetros) se toma con validación; el test se evalúa una sola vez al final.

Un error que enseñó algo. En las primeras pruebas con G8, la CNN acertaba el 90 % en train pero en validación respondía siempre la misma clase. La causa era BatchNorm: al predecir usa medias móviles que con el *momentum* por defecto de Keras (0,99) necesitan cientos de pasos para estabilizarse, y G8 tiene unos 10 pasos por época. Con *momentum* 0,9 el F1 en validación pasó de 0,33 a 0,91.

4. Arquitecturas

4.1 CNNs desde cero

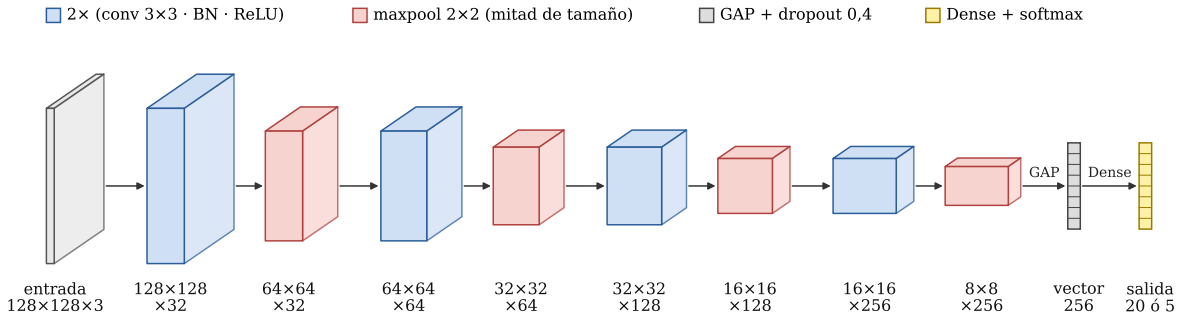
Se diseñaron cuatro CNNs chicas, cada una para probar una idea distinta. Las cuatro reciben 128×128×3 y terminan igual: *Global Average Pooling* (promedia cada mapa a un número, en lugar de aplanar y usar millones de pesos), dropout 0,4 y una densa con softmax (20 salidas en identidad, 5 en emoción). Todas las convoluciones van seguidas de BatchNorm y ReLU.

En los diagramas, cada cubo es el mapa de características que sale de esa etapa: el alto representa el tamaño espacial y el ancho la cantidad de canales. Al avanzar, la imagen se achica y los canales crecen: la red cambia resolución por cantidad de patrones distintos que puede detectar.

`cnn`: tipo VGG (1,18 M parámetros, 8 convoluciones)

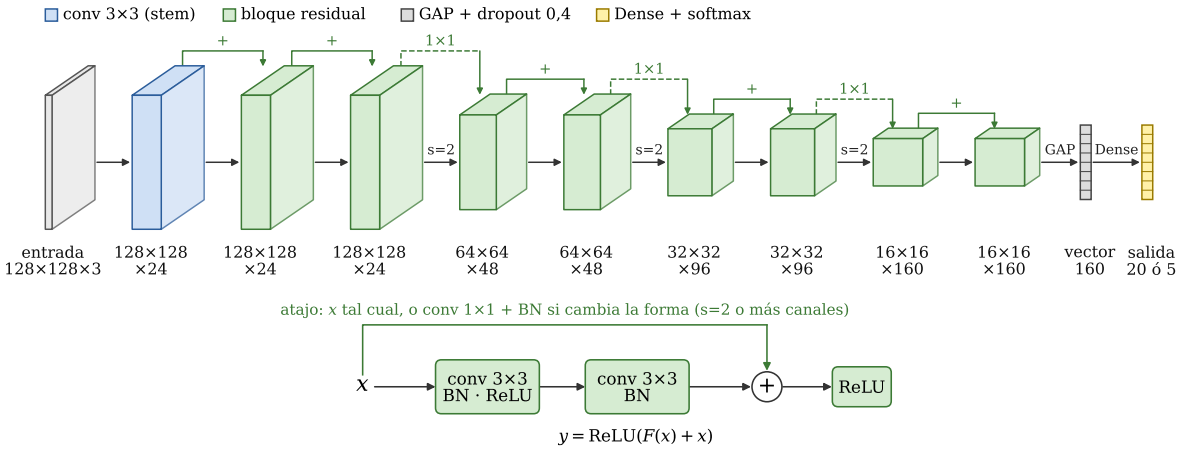
Cuatro bloques de dos convoluciones 3×3 y un maxpool que divide el tamaño a la mitad; los canales se duplican en cada bloque (32, 64, 128, 256). Es la referencia simple. Casi todos los parámetros están en el último bloque (887 mil de

1,18 M), porque una convolución cuesta $9 \cdot C_{\text{entrada}} \cdot C_{\text{salida}}$.



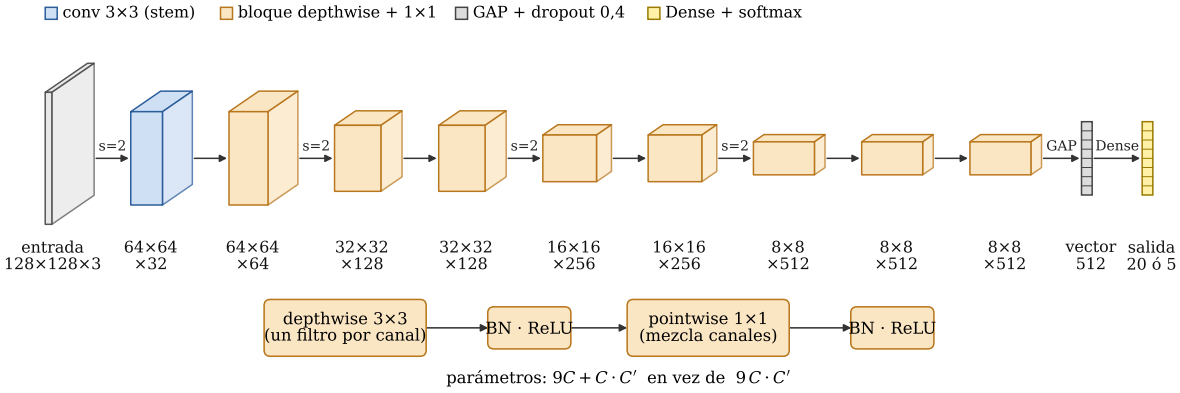
`cnn_residual`: mini-ResNet (1,24 M, 20 convoluciones)

Ocho bloques residuales en cuatro etapas. Cada bloque suma su entrada a la salida, $y = \text{ReLU}(F(x) + x)$: el gradiente pasa por el atajo sin atravesar las convoluciones, lo que permite más del doble de profundidad sin que el entrenamiento se degrade. No hay maxpool: la resolución baja con convoluciones de paso 2 (*s=2*), y en esos bloques el atajo usa una convolución 1×1 (línea punteada) para que las formas coincidan antes de sumar.



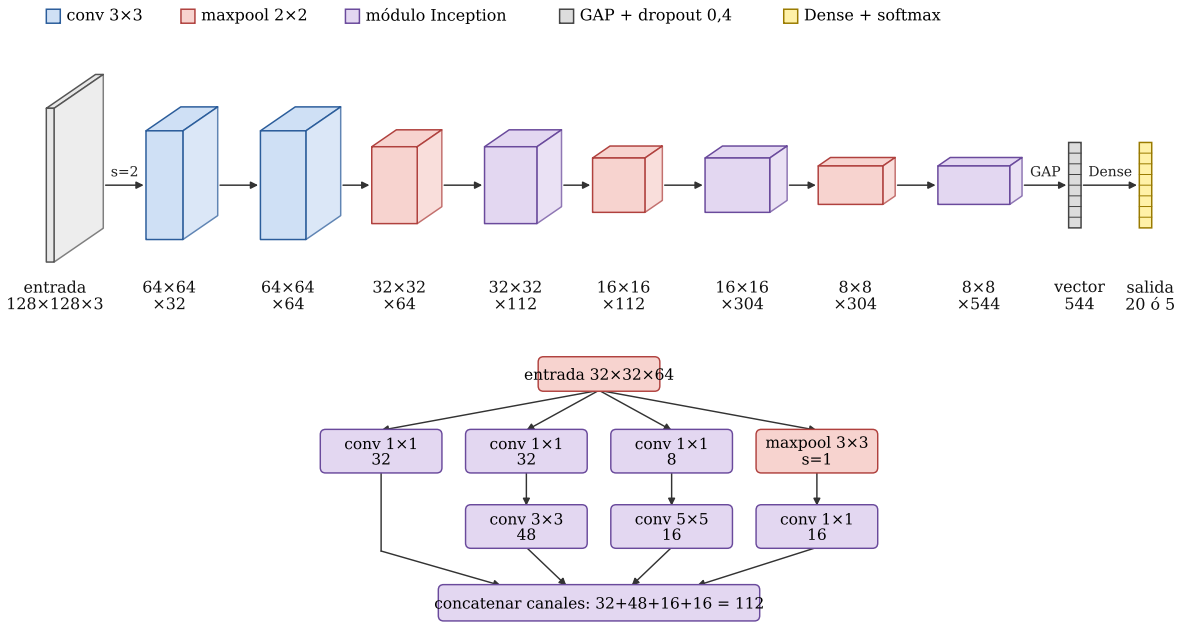
`cnn_separable`: mini-MobileNet (0,83 M, 17 convoluciones)

Cada bloque separa la convolución en una 3×3 por canal (*depthwise*) y una 1×1 que mezcla canales (*pointwise*). Para 256 canales de entrada y salida, una convolución normal cuesta $9 \cdot 256 \cdot 256 \approx 590$ mil parámetros y la separable $9 \cdot 256 + 256 \cdot 256 \approx 68$ mil: casi 9 veces menos. Con unas 1900 imágenes de train, menos parámetros significa menos capacidad de memorizar.



`cnn_inception`: mini-Inception (0,57 M, 20 convoluciones)

Cada módulo mira la misma entrada con ramas 1×1, 3×3, 5×5 y maxpool en paralelo y concatena los resultados en canales: en un mismo paso ve detalles finos (comisuras, cejas) y formas más grandes. El entrenamiento decide qué escala usar. Las convoluciones 1×1 previas reducen los canales para que las ramas grandes no multipliquen los parámetros: es la más chica de las cuatro.



Detalle del módulo 1 (32×32×64 → 32×32×112); los módulos 2 y 3 tienen 304 y 544 canales de salida.

4.2 Transfer learning

Se reutilizan redes entrenadas en ImageNet (1,2 millones de fotos, 1000 clases), cuyas capas ya detectan bordes, texturas y partes de objetos. Se les quita la cabeza original y se agrega una nueva para nuestras clases.

- **MobileNetV2** (2,3 M): liviana, con bloques residuales invertidos y convoluciones separables.
- **EfficientNetB0** (4,1 M): escala profundidad, ancho y resolución de forma balanceada.
- **ResNet50V2** (23,6 M): 50 capas con bloques residuales; la referencia grande.

4.3 Baselines

Para saber si un número es bueno hace falta un piso con qué compararlo:

- **Dummy:** responde siempre la clase más frecuente.
- **Random Forest** sobre los píxeles en escala de grises a 48×48: un modelo clásico sin redes.
- **Solo persona** (solo emoción): mira quién es y responde la emoción más frecuente de esa persona en train. Mide cuánto se puede sacar con el atajo persona → emoción.

Se descartaron RNN, LSTM y GRU: sirven para secuencias (texto, audio), y acá cada foto es independiente y tiene una sola etiqueta.

5. Métricas

- **Exactitud** (*accuracy*): proporción de aciertos. Con clases desbalanceadas es engañosa.
- **Precisión** de una clase: de las veces que el modelo dijo esa clase, cuántas eran correctas. **Recall**: de las imágenes de esa clase, cuántas encontró.
- **F1** de una clase: media armónica de precisión y recall, $F_1 = 2PR/(P+R)$. Solo es alto si las dos son altas.

- **F1 macro**: promedio simple del F1 de cada clase. Una clase chica pesa lo mismo que una grande, así que un modelo que ignora a Julio Chávez o a la tristeza queda penalizado. Es la métrica para elegir modelos.

El Dummy lo muestra bien: en emoción tiene 27,5 % de exactitud (acierta todos los neutrales) pero 0,086 de F1 macro.

6. Resultados

6.1 Comparación de modelos

			Identidad		Emoción	
Modelo	Tipo	Parám.	Val	Test	Val	Test
Dummy	baseline	—	—	0,013	—	0,086
Solo persona	baseline	—	—	—	—	0,255
Random Forest	baseline	—	—	0,842	—	0,360
`cnn`	desde cero	1,18 M	0,953	0,959	0,365	0,404
`cnn_residual`	desde cero	1,24 M	0,948	0,955	0,373	0,368
`cnn_separable`	desde cero	0,83 M	0,948	0,945	0,395	0,425
`cnn_inception`	desde cero	0,57 M	0,941	0,959	0,439	0,449
MobileNetV2	transferencia	2,3 M	0,972	0,961	0,449	0,464
EfficientNetB0	transferencia	4,1 M	0,970	0,951	0,390	0,459
ResNet50V2	transferencia	23,6 M	**0,976**	**0,974***	**0,463**	**0,482***

Cuadro 2. F1 macro en validación ($n = 413$) y test ($n = 411$) del dataset final. * Modelo elegido por validación. Con una sola semilla, diferencias de $\pm 0{,}02$ no son significativas.

Identidad. Todos los modelos entrenados quedan entre 0,95 y 0,97, y hasta el Random Forest llega a 0,84: con 20 personas y fotos de pocos videos la tarea es fácil. ResNet50V2 es la mejor, pero la diferencia es chica: la CNN Inception iguala a la VGG con la mitad de parámetros y a ResNet50V2 con 40 veces menos. En test, ResNet50V2 se equivoca en 11 de 411 imágenes; la confusión más repetida es Tini Stoessel → Lali Espósito (2 casos), dos personas que se parecen y aparecen en contextos similares.

Emoción. Es mucho más difícil. Gana el transfer learning (0,46–0,48): las capas preentrenadas ya distinguen formas finas como una boca abierta o cejas levantadas. Entre las CNNs desde cero rinden mejor las de menos parámetros (Inception 0,449 y separable 0,425 frente a 0,37–0,40), coherente con la idea de que con pocas imágenes conviene menos capacidad. Todos los modelos superan con claridad al baseline "solo persona".

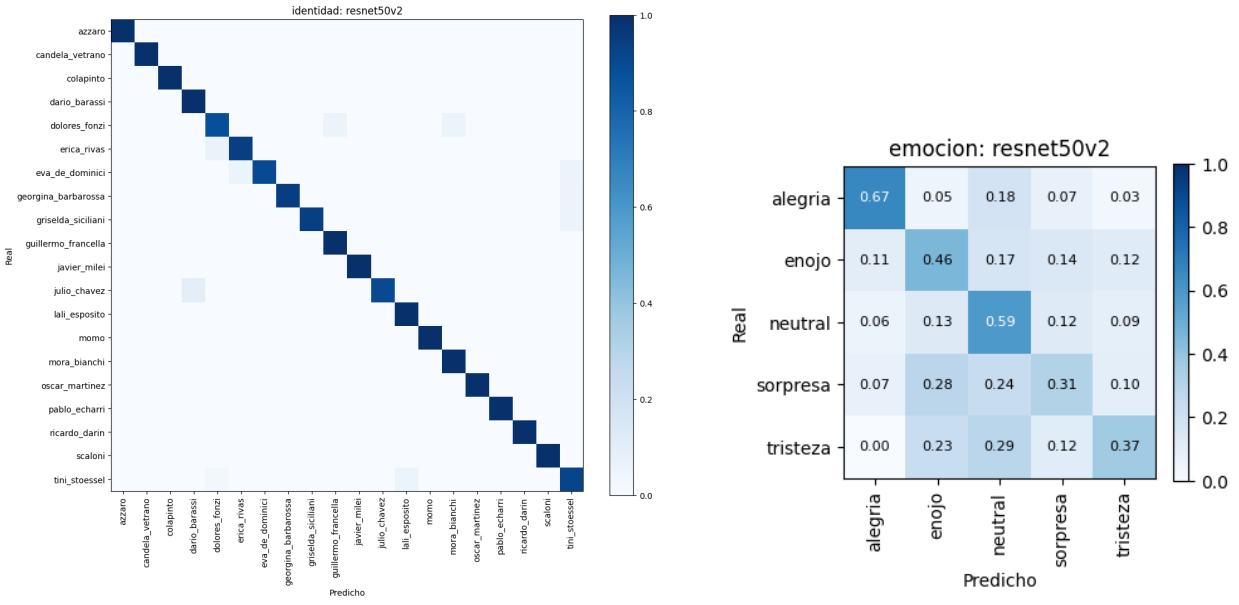


Figura 3. Matrices de confusión en test con ResNet50V2, normalizadas por fila. Izquierda: identidad. Derecha: emoción.

6.2 Errores en emoción

La matriz de emoción muestra que alegría es la más fácil (67 % de acierto: la sonrisa es muy marcada) y sorpresa la más difícil (31 %). Las confusiones más frecuentes son sorpresa → enojo (20), alegría → neutral (18), sorpresa → neutral (17), neutral → enojo (15) y tristeza → neutral (15). Tienen sentido: sorpresa y enojo comparten boca abierta y ojos tensos, y "neutral" absorbe los gestos suaves. Parte de estos errores probablemente sean etiquetas discutibles: en un fotograma de video la emoción no siempre es clara.

6.3 G8 dentro del test final

Las 34 imágenes de test de Fonzi y Echarri dan 94 % de acierto en identidad (F1 0,969 sobre esas dos clases) y 47 % en emoción (F1 macro 0,366), en línea con el resto del dataset.

7. ¿Sobreajuste o subajuste?

En la mejor época de cada entrenamiento se midió la **brecha** entre la exactitud de train y la de validación, y si la pérdida de validación sube después de su mínimo. Criterio:

- *Sobreajuste*: brecha mayor a 0,10 y pérdida de validación que sube (o brecha mayor a 0,20). El modelo memoriza train.
- *Subajuste*: exactitud de train menor a 0,70. El modelo no aprende ni siquiera los datos que ve.

	Identidad		Emoción		
Modelo	Train / Val	Brecha	Train / Val	Brecha	Diagnóstico emoción
`cnn`	0,98 / 0,95	0,04	0,51 / 0,38	0,13	sobreajuste
`cnn_residual`	0,98 / 0,94	0,04	0,51 / 0,38	0,13	sobreajuste
`cnn_separable`	1,00 / 0,95	0,05	0,56 / 0,41	0,15	sobreajuste
`cnn_inception`	0,99 / 0,93	0,06	0,62 / 0,48	0,14	sobreajuste
MobileNetV2	0,99 / 0,96	0,02	0,60 / 0,47	0,13	subajuste
EfficientNetB0	0,98 / 0,96	0,02	0,58 / 0,40	0,18	subajuste
ResNet50V2	1,00 / 0,97	0,03	0,68 / 0,48	0,20	subajuste

Cuadro 3. Exactitud de train y validación en la mejor época. En identidad todos los diagnósticos son "razonable" (`cnn_residual` y ResNet50V2 llegaron al máximo de épocas todavía mejorando décimas).

- **Identidad: sin sobreajuste preocupante.** Brecha de 2 a 6 puntos. Es lo esperable con 20 clases.
- **Emoción, CNNs desde cero: sobreajuste.** Después de la mejor época la pérdida de train sigue bajando y la de validación sube (en `cnn_separable`, train baja de 1,08 a 0,43 y validación sube de 1,49 a 1,79). El *early stopping* restaura la mejor época, así que el modelo guardado no es el sobreajustado.
- **Emoción, transfer learning: subajuste.** La brecha también es grande, pero la exactitud de train es solo 0,58–0,68: el modelo no logra separar bien las emociones ni en las imágenes con las que entrena.

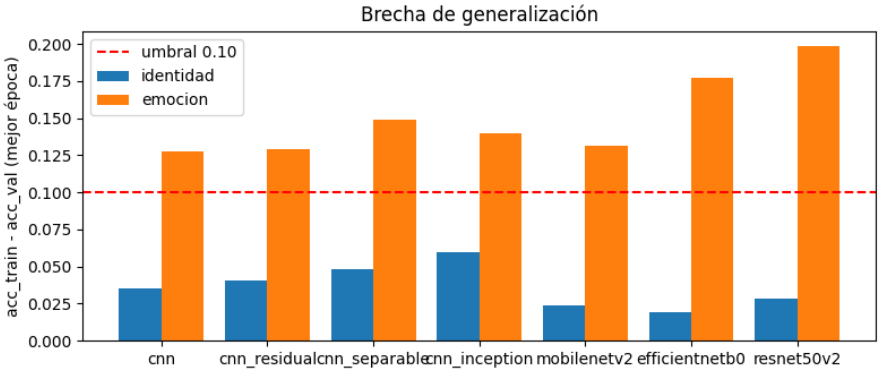


Figura 4. Brecha train – validación en la mejor época. La línea roja es el umbral de 0,10.

Conclusión. La emoción está limitada por los datos más que por la red: unas 1900 imágenes de train para 5 clases, gestos sutiles y etiquetas discutibles. Más regularización bajaría la brecha de las CNNs desde cero, pero el *early stopping* ya la controla y los mejores modelos son los de transferencia, que no memorizan. En una prueba previa, duplicar las épocas no mejoró el test. Lo que sí ayudaría es recortar más ajustado a la cara y revisar etiquetas.

8. ¿Qué aprende el modelo?

8.1 Qué ve la CNN por dentro

Las activaciones de la `cnn` muestran cómo cambia la representación con la profundidad. El primer bloque (64×64) responde a bordes y contornos: todavía se reconoce la cara. En los bloques siguientes la resolución baja y los mapas pasan a marcar regiones (ojos, boca, pelo, fondo) hasta que en el último (8×8) cada canal es un detector de un patrón abstracto en una zona aproximada.

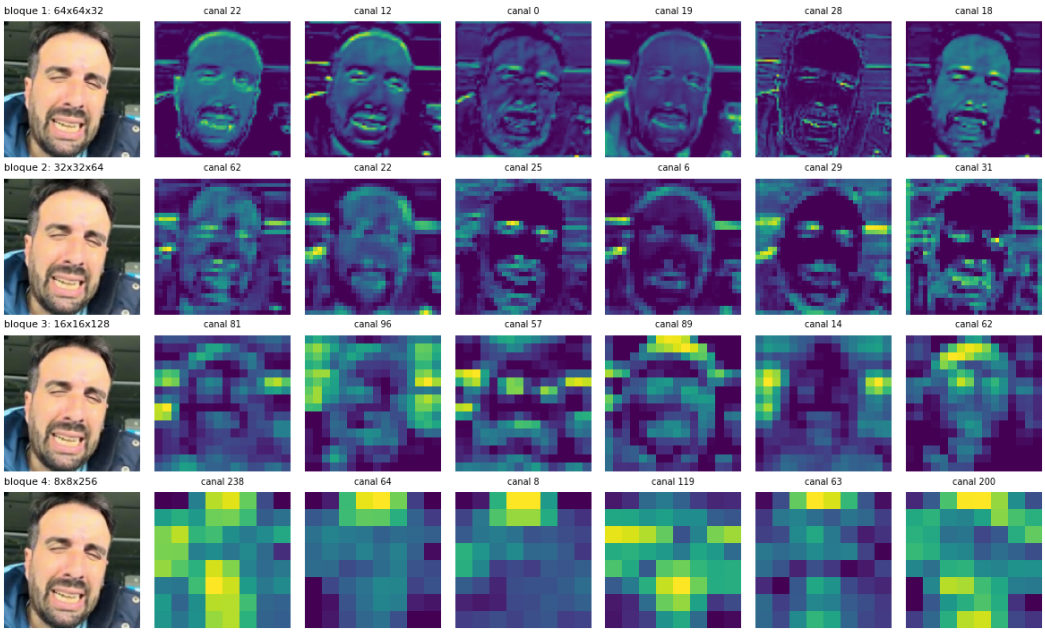


Figura 5. Seis canales de cada bloque de la `cnn` (identidad) para la misma imagen.

8.2 Atajo persona → emoción

Si el modelo de emoción reconociera a la persona y respondiera su emoción habitual, tendría un número aceptable sin mirar el gesto. Tres pruebas indican que el atajo existe pero es parcial:

- El baseline "solo persona" llega a 0,255 (Dummy 0,086): el atajo da algo. Pero ResNet50V2 llega a 0,482, casi el doble.
- En las 273 imágenes de test (66 %) cuya emoción no es la habitual de su persona, el modelo acierta el 39 %, repite la emoción habitual el 23 % y se equivoca hacia otra el 37 %.
- El rendimiento por persona varía mucho (de 0,27 de exactitud en Darío Barassi a 0,68 en Eva De Dominici), lo que también refleja que algunas personas son más expresivas o tienen fotos más claras.

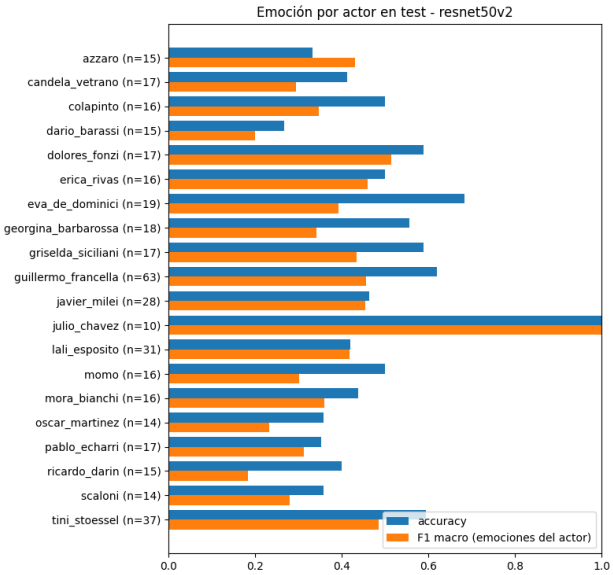


Figura 6. Exactitud y F1 macro de emoción por persona en test (ResNet50V2). Julio Chávez tiene solo 10 imágenes.

8.3 Grad-CAM: dónde mira la red

Grad-CAM arma un mapa de calor de las zonas que más influyeron en la predicción. Para la clase c y los mapas A^k de la última capa convolucional, se calcula cuánto cambia la puntuación de la clase con cada mapa y se combinan:

$$\alpha_k = \frac{1}{Z} \sum_{i,j} \frac{\partial y^c}{\partial A^k_{ij}}, \qquad L^c = \mathrm{ReLU}\Big(\sum_k \alpha_k A^k \Big).$$

Para resumirlo en un número se midió qué fracción del mapa cae en el 60 % central de la imagen, donde está la cara (un mapa uniforme da 0,36):

- **Identidad: 0,461.** Mira la cara, pero también pelo, ropa, zócalos y textos del video. Es una señal de que parte del acierto viene del contexto.
- **Emoción: 0,395** en general, 0,420 en los aciertos y 0,369 en los errores: cuando acierta mira más la cara (boca y ojos), y cuando se equivoca se va al fondo.



Figura 7. Grad-CAM en test con ResNet50V2. Arriba: identidad. Abajo: emoción. En cada grilla, la primera fila son aciertos y la segunda errores; el recuadro marca la zona central.

8.4 Fuga de escena

El phash agrupa solo fotogramas casi idénticos; dos fotogramas del mismo video separados por unos segundos pueden diferir lo suficiente para caer en grupos distintos y terminar uno en train y otro en test. El modelo puede entonces reconocer el fondo, la ropa o la iluminación de esa escena. Una partición por escena del mismo dataset da cifras más bajas (identidad ≈0,87, emoción ≈0,37), así que los resultados de la tabla están inflados por este efecto. Es coherente con lo que muestra Grad-CAM.

8.5 Calidad del dataset con un autoencoder

Se entrenó un autoencoder convolucional con ruido (entrada de 64×64, espacio latente de 64 dimensiones) que aprende a comprimir y reconstruir las imágenes. Sirve para dos cosas:

- **Detectar recortes raros:** las imágenes peor reconstruidas (error alto) son las más distintas del resto, candidatas a revisar a mano (recortes descentrados, caras muy chicas, fondos dominantes). El error medio es parecido en las tres particiones (0,012 train, 0,016 val, 0,015 test): no hay una partición “rara”.
- **Ver si las clases se separan sin etiquetas:** en el espacio latente casi no se separan ni personas (silueta 0,021 contra $-0,028$ con etiquetas al azar) ni emociones ($-0,022$). Lo que agrupa el autoencoder es sobre todo color, iluminación y fondo; las diferencias que importan son finas y las aprenden las redes supervisadas.

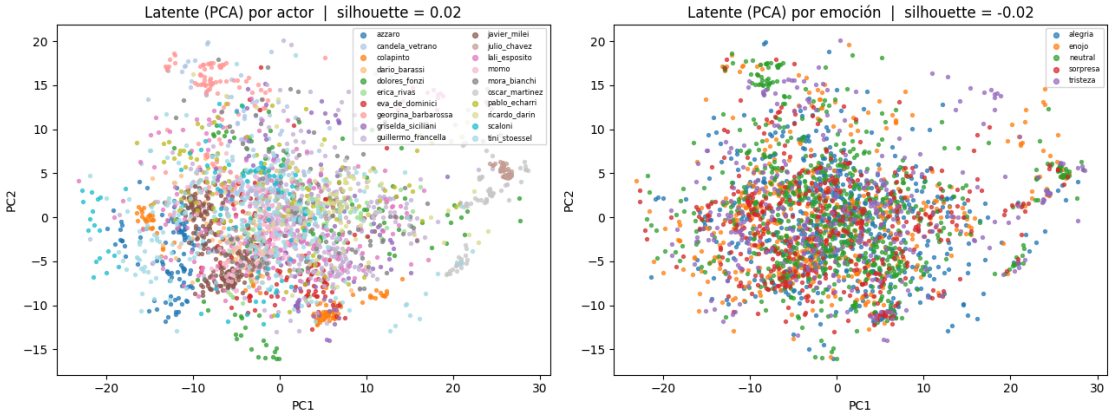


Figura 8. Espacio latente del autoencoder proyectado con PCA. Izquierda: color por persona. Derecha: por emoción.

9. Aplicación web

Los modelos se exportaron a TorchScript (desde la réplica en PyTorch del mismo pipeline, entrenada con las mismas decisiones) y se publicaron en una aplicación de Streamlit. Permite elegir al azar una foto de test o subir una propia, y muestra las tres personas y las tres emociones más probables, indicando si acertó. La foto se recorta al cuadrado central y se lleva a 224×224, igual que en el entrenamiento. Con fotos propias rinde mejor si la cara ocupa casi toda la imagen; como todo clasificador cerrado, siempre elige a una de las 20 personas, aunque la foto sea de otra.

10. Conclusiones

- **Identidad** se resuelve muy bien (F1 macro 0,97) con cualquier CNN razonable; una red chica bien diseñada (Inception, 0,57 M) está a la par de una de 23 M. Parte del acierto viene del contexto del video.
- **Emoción** es el problema difícil (0,48). El transfer learning ayuda, pero el techo lo ponen los datos: pocas imágenes, gestos sutiles y etiquetas ambiguas. Las CNNs desde cero sobreajustan y las preentrenadas no llegan a aprender train.
- Los controles (baseline “solo persona”, Grad-CAM, partición por escena) muestran que los números son optimistas y explican por qué. Vale más un número honesto que uno alto.

Próximos pasos: adoptar una partición por escena para todo el grupo y reentrenar; recortar más ajustado al rostro y revisar los recortes que marcó el autoencoder; medir el acuerdo entre etiquetadores de emoción; repetir con varias semillas para tener intervalos.